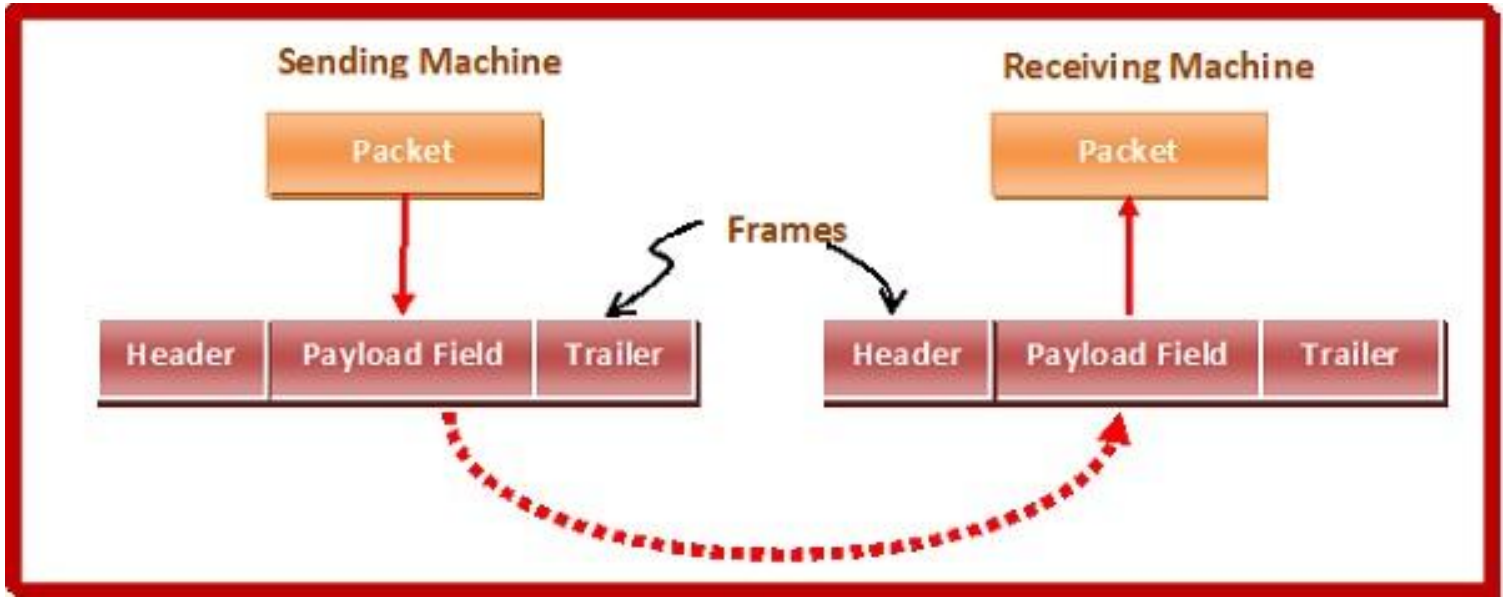
Data Link Layer

- The data link layer adds **a layer 2 header and a layer 2 trailer** to the packet it receives from the Network Layer (Layer 3).
- When you send a letter to someone in another country, you just drop it in your local mailbox. You don't worry about how it will travel all the way there. Behind the scenes, the postal system moves your letter **from one post office to another**.
- The data link layer works the same way. It handles how data is moved **from one device to the next directly connected device,** just like how a letter moves from one post office to the next. It adds a "**wrapper**" around the data (called a frame) to help with delivery over that specific link.
- It is responsible for receiving and getting data bits usually from Physical Layer and then converting these bits into groups, **known as data link frames** so that it can be transmitted further. It is also responsible to handle errors that might arise due to transmission of bits.

Data Link Layer

- In the data link layer, *framing* refers to the encapsulation of data to be sent over a network into multiple frames. Upon receiving a packet of information, the data link layer divides it into a sequence of bits known as a *frame*.
- Each frame has its own header and trailer, as well as a specific syntax or structure. This enables the data receiver to identify where the frame starts and ends. The header and trailer also enable receivers to distinguish between different frames.



Data Link Layer

Parts of a Frame:

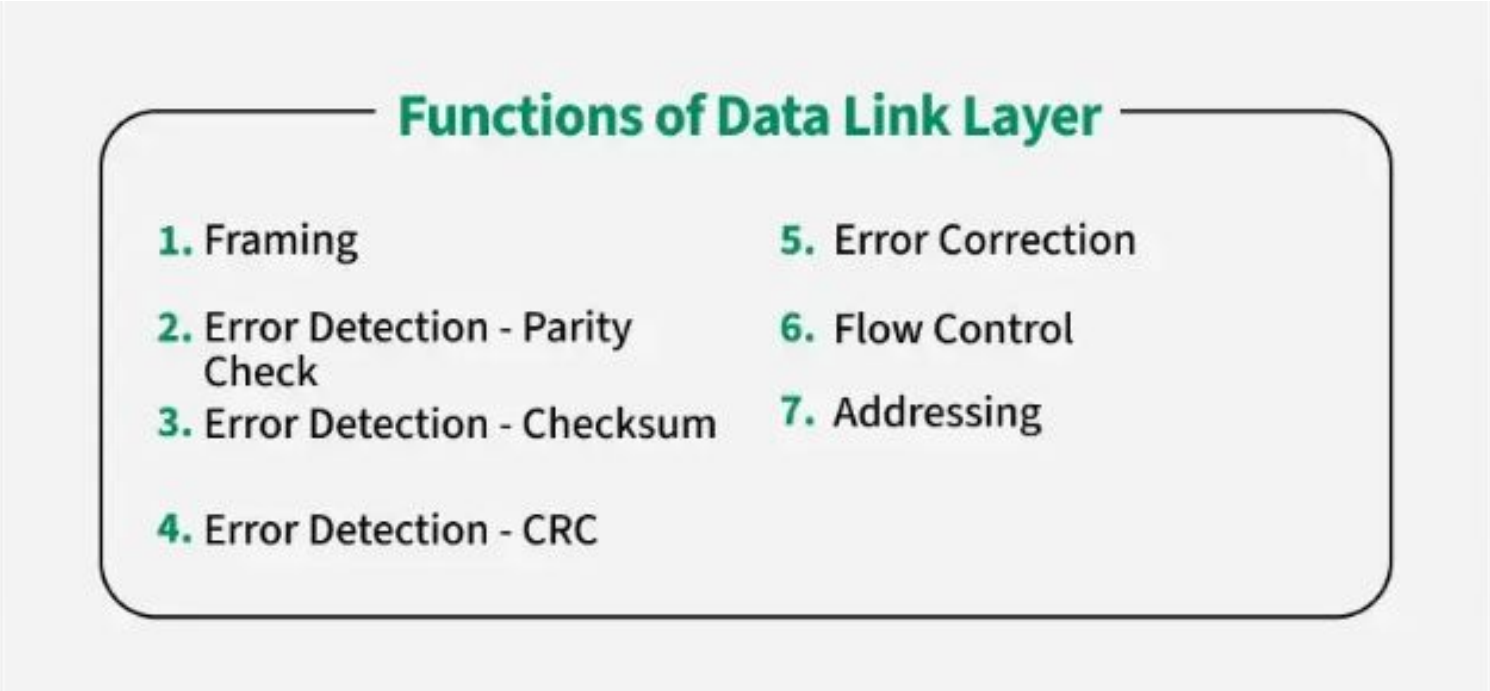
A frame has the following parts-

- **Frame Header** It contains the source and the destination addresses of the frame.
- **Payload field** It contains the message to be delivered.
- **Trailer** It contains the error detection and error correction bits.
- **Flag** It marks the beginning and end of the frame.



Services Offered by Data Link Layer

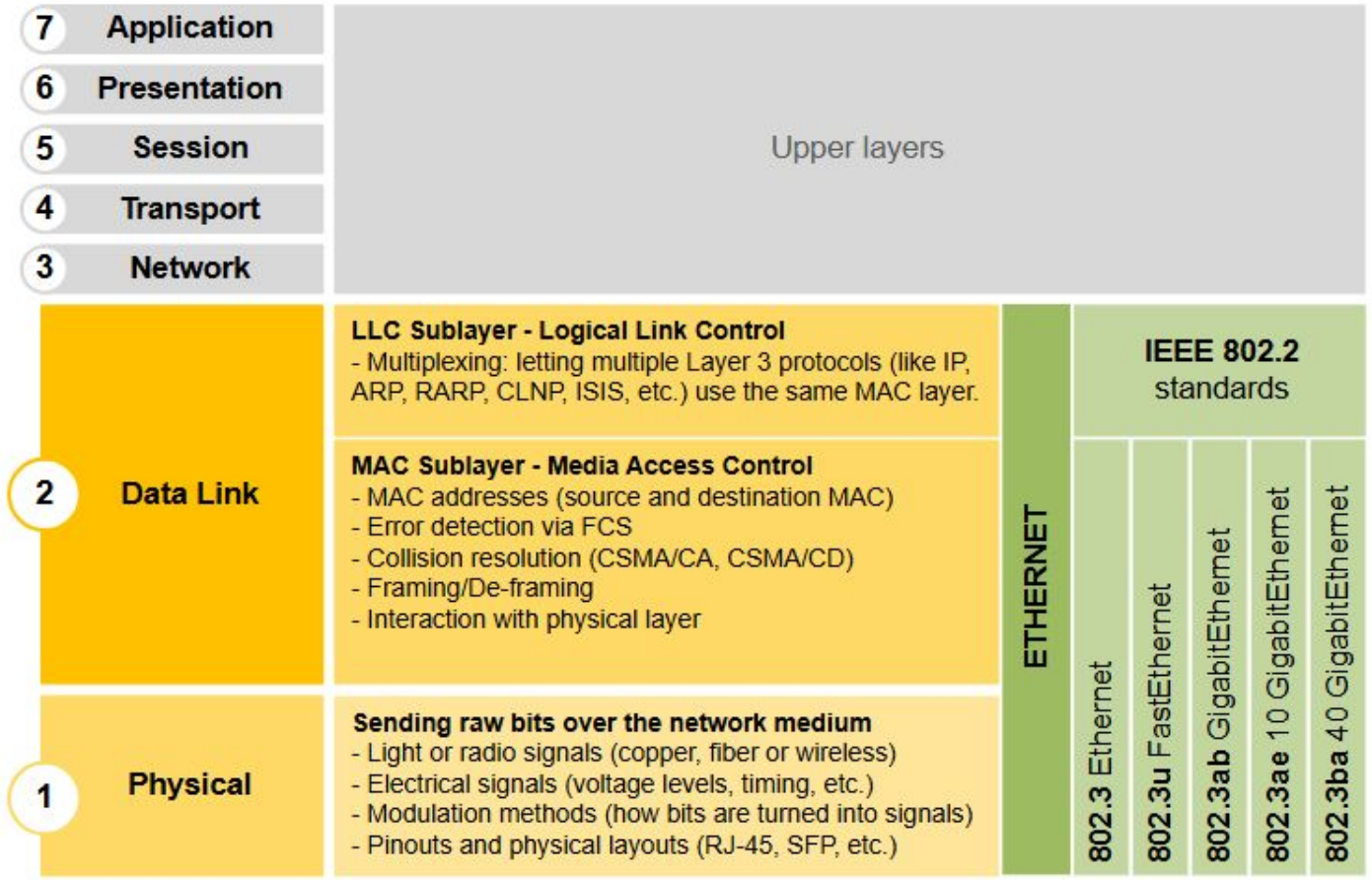
The data link layer provides various services as given below :



Sublayer

 The data link layer has two sublayers, as shown in the diagram below:

1. Logical Link Control (LLC) sublayer.
2. Media Access Control (MAC) sublayer.



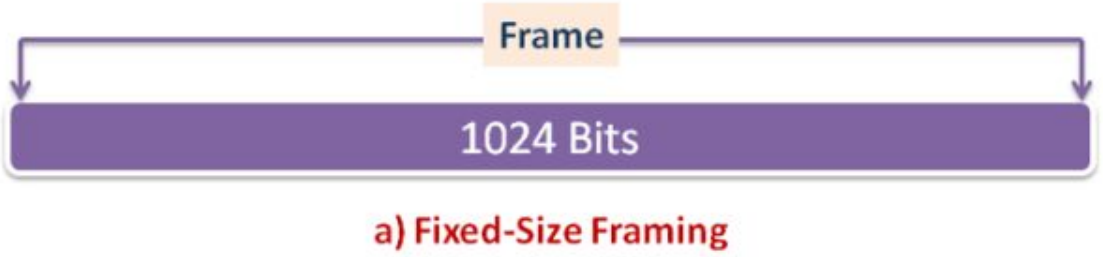
Sublayer

Here's how the two sublayers of the data link layer enable data transmission:

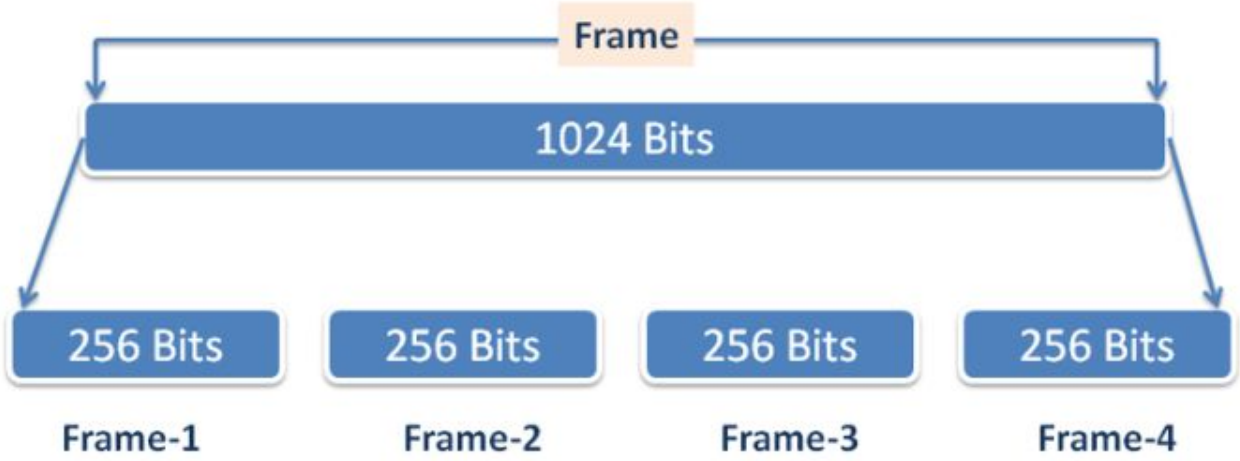
- The LLC sublayer delivers data to the MAC sublayer.
- The MAC sublayer adds the source and destination MAC addresses to the data.
- It stores the data in a buffer.
- It also forms an Ethernet frame and sends it to the destination node after checking its MAC address.
- The destination node checks the MAC address table to confirm the destination MAC address is present. If it finds a matching entry, it accepts the frame. Otherwise, it discards the frame.

Framing

- The data link layer organizes raw bits from the physical layer into structured data units called frames. It ensures proper synchronization between sender and receiver by adding headers and trailers to the data. Framing helps in identifying the beginning and end of a data packet and hence, preventing data loss or corruption.
- There are two types of framing:
  - Fixed size: frames have a predefined size.
  - Variable size: frame may be different size.



a) Fixed-Size Framing



b) Variable-Size Framing

Framing

Variable Sized Framing

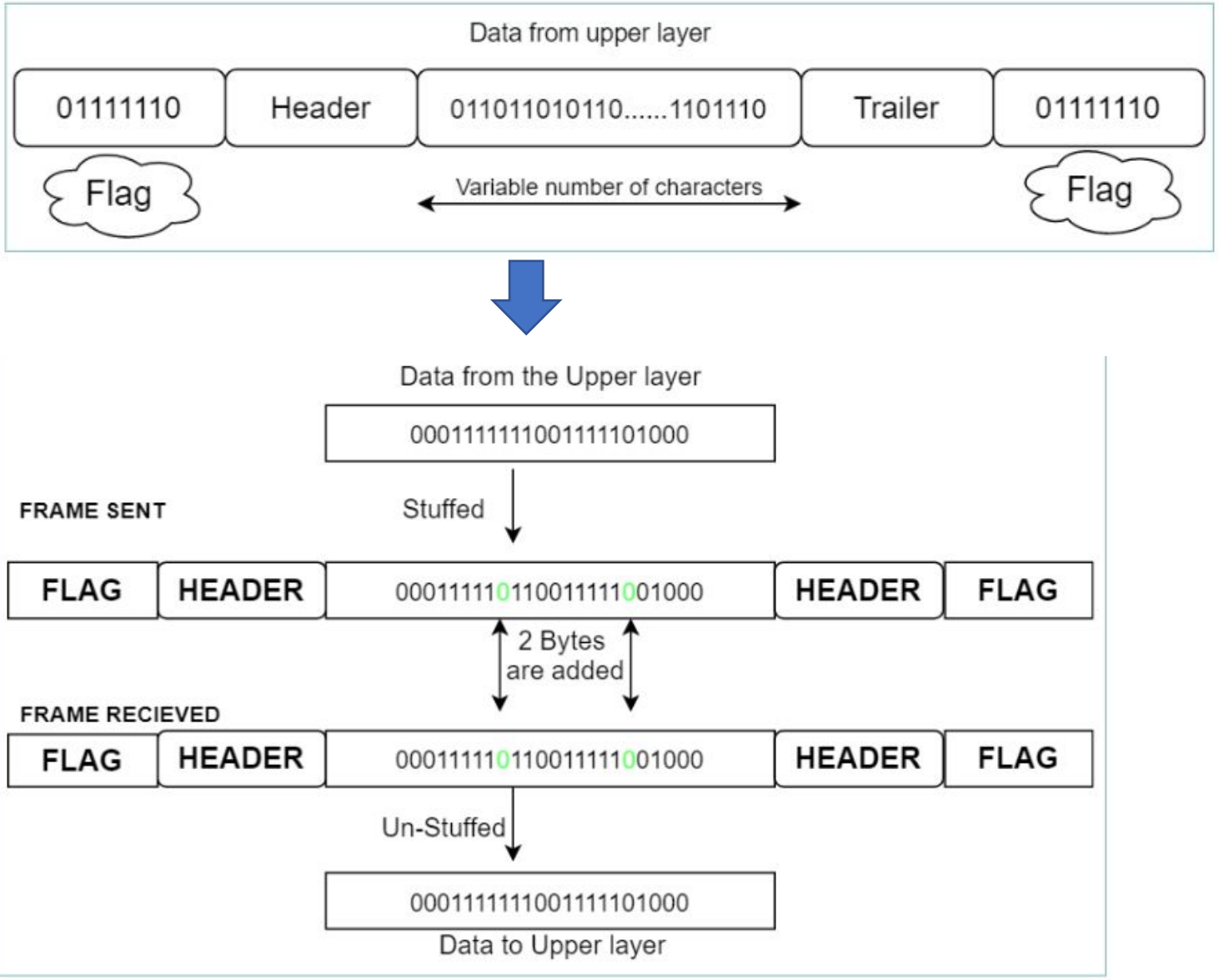
- Here, the size of each frame to be transmitted may be different. So additional mechanisms are kept to mark the end of one frame and the beginning of the next frame.
- Two ways to define frame delimiters in variable sized framing are-

- **Length Field** Here, a length field is used that determines the size of the frame. It is used in Ethernet (IEEE 802.3).
- **End Delimiter** Here, a pattern is used as a delimiter to determine the size of frame. It is used in Token Rings. If the pattern occurs in the message, then two approaches are used to avoid the situation
  - **Byte Stuffing:** A byte is stuffed in the message to differentiate from the delimiter. This is also called character-oriented framing.
  - **Bit Stuffing**: A pattern of bits of arbitrary length is stuffed in the message to differentiate from the delimiter. This is also called bit oriented framing.

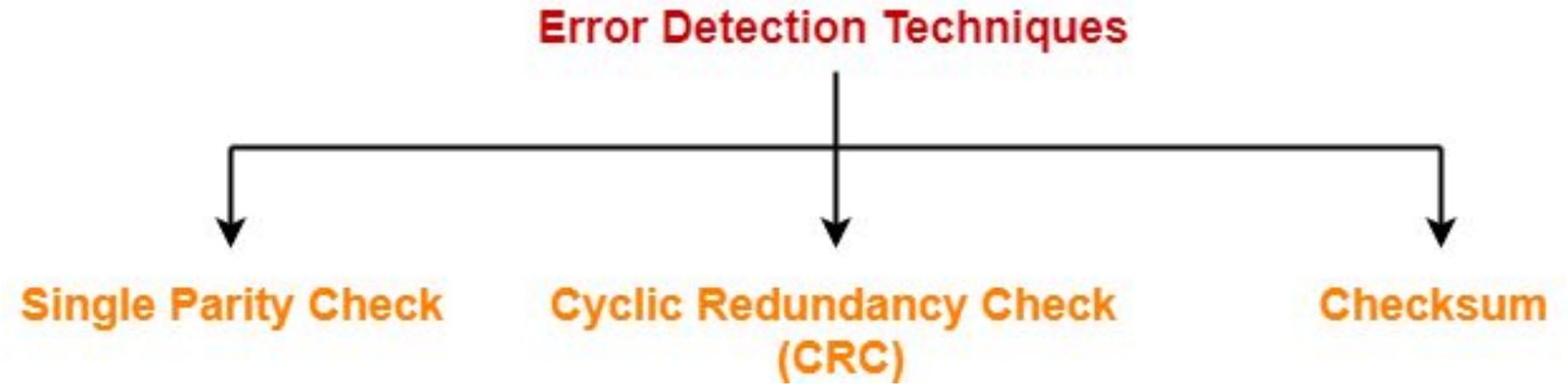
Framing

- Data frames can contain an arbitrary number of bits which allows the character codes with an arbitrary number of bits at each character, where each special frame begins and ends with a special bit pattern.
- Let's say 01111110, a flag byte. We know that when we are sending data in the payloads so at the beginning and the end we are having a flag byte which is 01111110.
- Whenever the sender's Data link layer encounters five consecutive 1's in the data, it automatically stuffs a 0 bit into the outgoing bitstream which is called **payloads.**
- If 6 1's come then at the receiving end it might get misinterpreted that it is an end flag, So after the occurrence of each consecutive 5 1's one zero will be stuffed & that's why it is called Bit stuffing.

Bit Stuffing



Error Detection



Cyclic Redundancy Check-

- Cyclic Redundancy Check (CRC) is an error detection method.
- It is based on binary division.

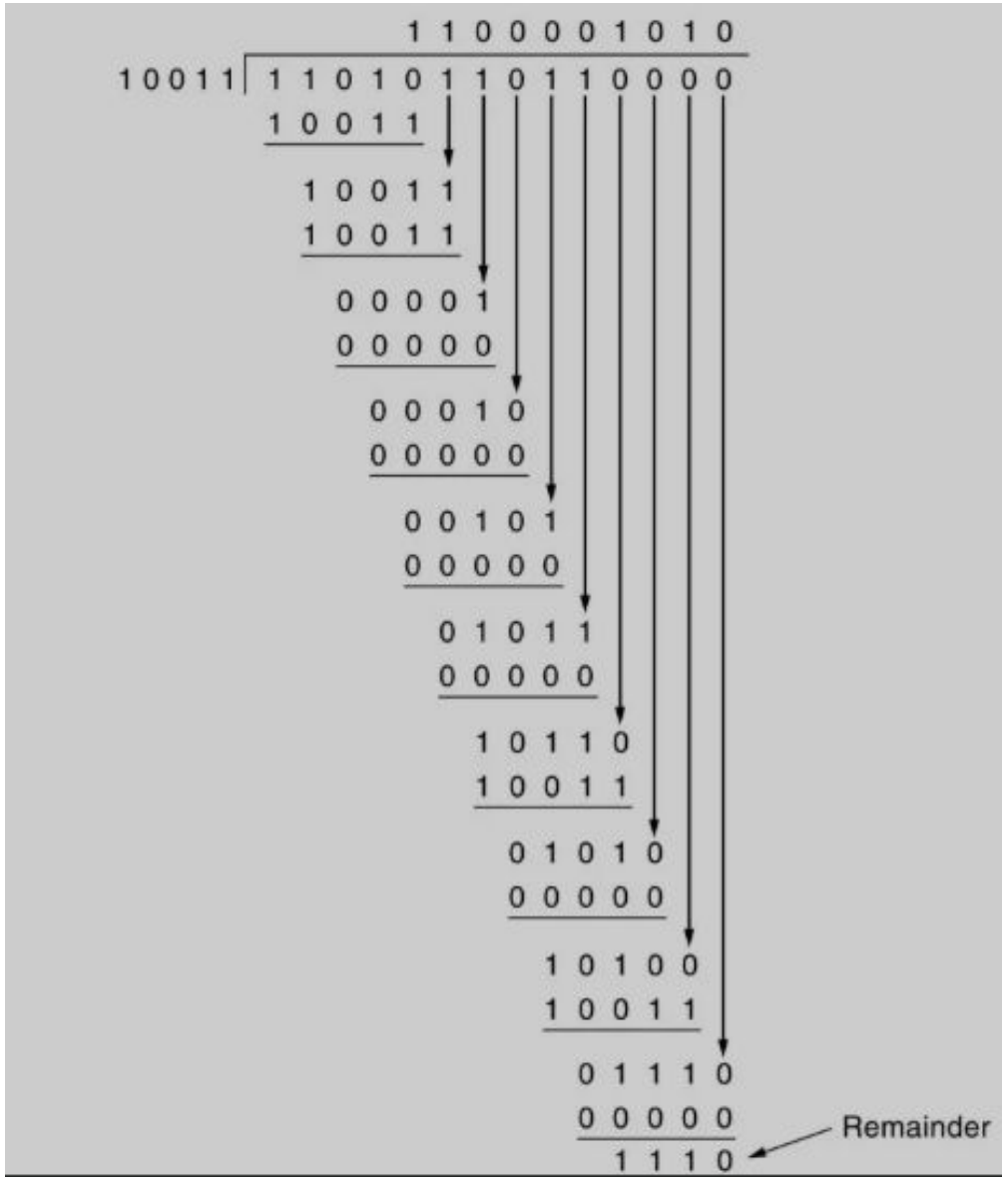
Error Detection

Problem: A bit stream 1101011011 is transmitted using the standard CRC method. The generator polynomial is x^4+x+1. What is the actual bit string transmitted?

Solution:

- The generator polynomial $G(x) = x^4 + x + 1$ is encoded as 10011 ($1.x^4+ 0.x^3+0.x^2+1.x^1+ 1. x^0$)
- Clearly, the generator polynomial consists of 5 bits.
- So, a string of 4 zeroes is appended to the bit stream to be transmitted.
- The resulting bit stream is 1101011011**0000**.

Error Detection



From here, CRC = 1110.

Now,

- The code word to be transmitted is obtained by replacing the last 4 zeroes of 1101011011**0000** with the CRC.
- Thus, the code word transmitted to the receiver = 1101011011**1110** **(actual bit string during transmission)**

- Now, if we divide 11010110111**1110** **by 10011**

10011) 11010110111**1110**

10011

10011

10011

00001

00000

00010

……

Continued

Al last the remainder will be **00000**

So, no problem occured during transmission

- Suppose, a bit has been inverted during the transmission. So the remained will not be 0. Thus, the receiver detect error.
- 1101011011**1110**
- 1101010011**1110**
- **Now if we divide** 1101010011**1110** by 10011 , the remainer will not be all zero.